

Lin_Timo_CSE 565_LoadTestingProject

Task 1 — Identification of Framework

I chose **k6** as the load-testing framework for this project. **k6** is an open-source tool designed for testing the performance and reliability of APIs and web services. It can generate large volumes of concurrent virtual users from a single command-line interface while remaining lightweight and scriptable. By writing test scripts in JavaScript, k6 allows the tester to define user behavior with precise control over concurrency, iteration rate, and runtime thresholds.

Reference

- <https://grafana.com/docs/k6/latest/>
- <https://grafana.com/docs/k6/latest/using-k6/http-requests/>
- <https://grafana.com/docs/k6/latest/using-k6/k6-options/>
- <https://grafana.com/docs/k6/latest/using-k6/thresholds/>

Task 2 — Test Case Design

Flask Wrapper

Flask endpoint that runs `execute_tasks()` once per request:

```
from flask import Flask, jsonify
import load_test_tasks as t

app = Flask(__name__)

@app.post("/execute")
def execute():
    try:
        result = t.execute_tasks()
        return jsonify({"ok": True, "result": result}), 200
    except Exception as e:
        return jsonify({"ok": False, "error": str(e)}), 500
```

Run server with worker(s):

```
gunicorn -w 8 -b 0.0.0.0:8000 app:app
```

k6 Script

```
import http from "k6/http";
import { check, sleep } from "k6";

export default function () {
  const res = http.post("http://localhost:8000/execute", null, {
    headers: { "Content-Type": "application/json" },
  });
  check(res, {
    "status is 200": (r) => r.status === 200,
    "body ok": (r) => r.json("ok") === true,
  });

  sleep(1);
}
```

Test Cases

Light load:

```
k6 run --vus 100 --duration 1m test.js
```

Medium load:

```
k6 run --vus 1000 --duration 2m test.js
```

Target load:

```
k6 run --vus 10000 --duration 5m test.js
```

Task 3 — Monitoring & Execution Report

Metrics

- Requests per second (RPS): system throughput
- Response time (avg, median, p95, p99): user experience and tail latency
- Failures in real time: stability and error modes

Light load with 100 users

```
http_req_duration.....: avg=7.95s min=768.12ms med=8.18s max=9.39s p(90)=8.76s p(95)=8.8s
{ expected_response:true }...: avg=7.95s min=768.12ms med=8.18s max=9.39s p(90)=8.76s p(95)=8.8s
http_req_failed.....: 0.00% 0 out of 764
http_reqs.....: 764 11.070548/s
```

Medium load with 1000 users

```

HTTP
http_req_duration.....: avg=51.49s min=725.48ms med=59.99s max=1m0s p(90)=1m0s p(95)=1m0s
{ expected_response:true }...: avg=30.23s min=725.48ms med=30.2s max=59.61s p(90)=53.87s p(95)=56.74s
http_req_failed.....: 71.41% 1656 out of 2319
http_reqs.....: 2319 15.45958/s

```

Large load with 10000 users

```

HTTP
http_req_duration.....: avg=7.72s min=0s med=0s max=1m0s p(90)=40.56s p(95)=54.84s
{ expected_response:true }...: avg=30.46s min=750.82ms med=30.63s max=59.82s p(90)=54.1s p(95)=56.97s
http_req_failed.....: 99.30% 95313 out of 95976
http_reqs.....: 95976 290.828207/s

```

Task 4 – Analysis

At **light load (100 users)**, the system was stable with no failures and consistent ≈ 8 s response times. This suggests that the combined tasks—sorting data, simulated database wait, API call, file I/O, computation, and logging—ran within capacity across the four Gunicorn workers.

At **medium load (1 000 users)**, the average response time spiked to ≈ 51 s and more than 70 % of requests failed. This indicates CPU and disk contention: the data-processing and computation functions are CPU-bound, while the file I/O task contends for disk throughput. Additionally, many failures likely originated in the `api_request()` function, where the external service (`jsonplaceholder.typicode.com`) throttles requests when hit concurrently.

At **large load (10 000 users)**, nearly all requests failed (≈ 99 %). The few that succeeded showed extremely long tail latencies ($p_{95} \approx 55$ s). This points to overall system saturation, which means process queues overflowed, threads starved for CPU time, and external HTTP connections began timing out. In short, the bottlenecks lie in CPU-intensive operations and network I/O to the third-party API, both of which scale poorly under massive concurrency.

Task 5 – Improvement of Test Strategy

After completing the baseline load tests in Task 2 and analyzing the results in Task 4, it was evident that the original testing approach, which was executing the combined workload through a single `/execute` endpoint, identified overall system degradation but did not clearly reveal which component caused the slowdown or failures. Therefore, the test strategy was improved to produce more detailed insights by isolating system components and applying targeted load conditions.

Revised Strategy

The Flask wrapper was modified to expose three additional endpoints (`/cpu`, `/io`, and `/api`) each responsible for a specific type of operation.

- `/cpu` executes the data-processing and computation tasks to stress the CPU.
- `/io` performs file-write/read and logging operations to stress the disk.
- `/api` sends requests to the external API to isolate network and remote-service performance. This isolation enables direct observation of how each subsystem behaves under increasing concurrency.

Implement Result

1000 users, /cpu

```
HTTP
http_req_duration.....: avg=8.46s min=64.31ms med=9.08s max=9.71s p(90)=9.62s p(95)=9.63s
{ expected_response:true }...: avg=8.46s min=64.31ms med=9.08s max=9.71s p(90)=9.62s p(95)=9.63s
http_req_failed.....: 0.00% 0 out of 13191
http_reqs.....: 13191 101.734293/s
```

1000 users, /io

```
HTTP
http_req_duration.....: avg=3.71ms min=410.3µs med=1.3ms max=132.04ms p(90)=8.36ms p(95)=12.62ms
{ expected_response:true }...: avg=3.71ms min=410.3µs med=1.3ms max=132.04ms p(90)=8.36ms p(95)=12.62ms
http_req_failed.....: 0.00% 0 out of 120000
http_reqs.....: 120000 994.888312/s
```

1000 users, /api

```
HTTP
http_req_duration.....: avg=16.88s min=226.08ms med=17.89s max=19.3s p(90)=18.61s p(95)=18.64s
{ expected_response:true }...: avg=16.88s min=226.08ms med=17.89s max=19.3s p(90)=18.61s p(95)=18.64s
http_req_failed.....: 0.00% 0 out of 7232
http_reqs.....: 7232 52.11966/s
```

10000 users, /cpu

```
HTTP
http_req_duration.....: avg=11.31s min=0s med=0s max=1m0s p(90)=26.55s p(95)=40.34s
{ expected_response:true }...: avg=21.96s min=106.55ms med=22.11s max=56.77s p(90)=26.65s p(95)=27.87s
http_req_failed.....: 71.46% 77142 out of 107944
http_reqs.....: 107944 327.094004/s
```

10000 users, /io

```
HTTP
http_req_duration.....: avg=2.17s min=0s med=302.95ms max=1m0s p(90)=2.98s p(95)=11.84s
{ expected_response:true }...: avg=1.83s min=465.17µs med=310.32ms max=58.08s p(90)=2.56s p(95)=11.26s
http_req_failed.....: 11.30% 47587 out of 421000
http_reqs.....: 421000 1275.751266/s
```

10000 users, /api

```
HTTP
http_req_duration.....: avg=11.75s min=0s med=0s max=59.95s p(90)=39.94s p(95)=46.66s
{ expected_response:true }...: avg=38.72s min=177.99ms med=39.39s max=57.22s p(90)=47.28s p(95)=47.87s
http_req_failed.....: 83.35% 82578 out of 99069
http_reqs.....: 99069 300.201592/s
```

Discussion

This refined strategy revealed three independent bottlenecks, which were CPU saturation, disk contention, and external API throttling, that were not clearly identifiable under the original all-in-one workload. By targeting each subsystem separately, it became possible to determine exactly when and where failures occur. The results demonstrate that CPU tasks scale poorly beyond several thousand concurrent requests, disk I/O quickly collapses under large parallel writes, and the external API cannot sustain heavy request volumes without rate limiting.

In conclusion, the improved strategy successfully identified additional performance bottlenecks by combining subsystem isolation with controlled arrival-rate testing. These refinements transformed a general scalability test into a diagnostic experiment that explains why performance degrades, not just that it does. The evidence supports future optimization recommendations such as parallelizing CPU operations, batching or caching file writes, and introducing retry and back-off logic for external API calls.